MoneyMate

How we built a personal AI-powered expense tracker

Project walkthrough | Architecture • Features • MCP • LangGraph • Deployment

October 2026 | Practical technical documentation

MoneyMate helps track balances and spending across multiple bank accounts. Its standout feature is a chat assistant that can answer financial questions using actual stored data and request approval before recording a transaction.

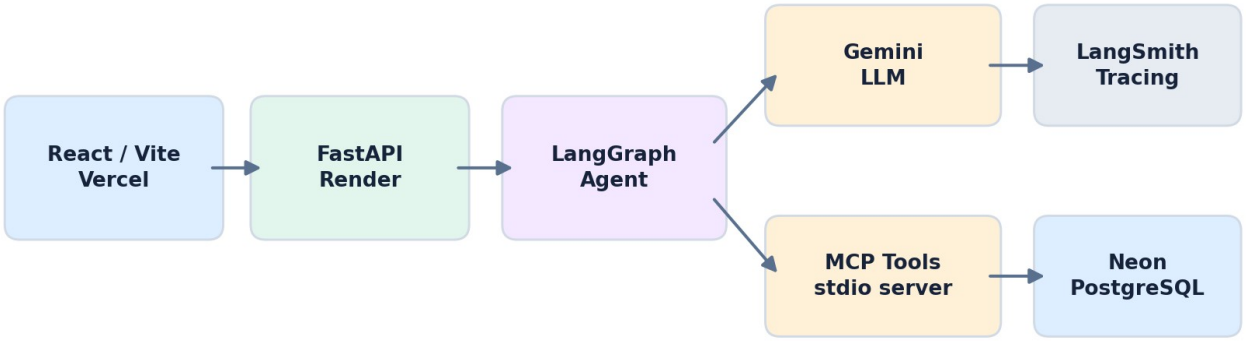


Figure 1. MoneyMate at a glance: a React frontend, FastAPI backend, AI agent, MCP tools and PostgreSQL database.

1. What problem does MoneyMate solve?

Managing several bank accounts manually makes it hard to know how much money is available, what was spent recently and which categories consume the most. MoneyMate combines a dashboard with a conversational assistant to make these tasks easier.

- Track multiple accounts and their current balances.
- Record expenses, income and transfers between accounts.
- View totals for selected reporting periods and spending by category.
- Ask natural-language questions instead of navigating several screens.
- Approve or cancel money-changing actions before they are saved.

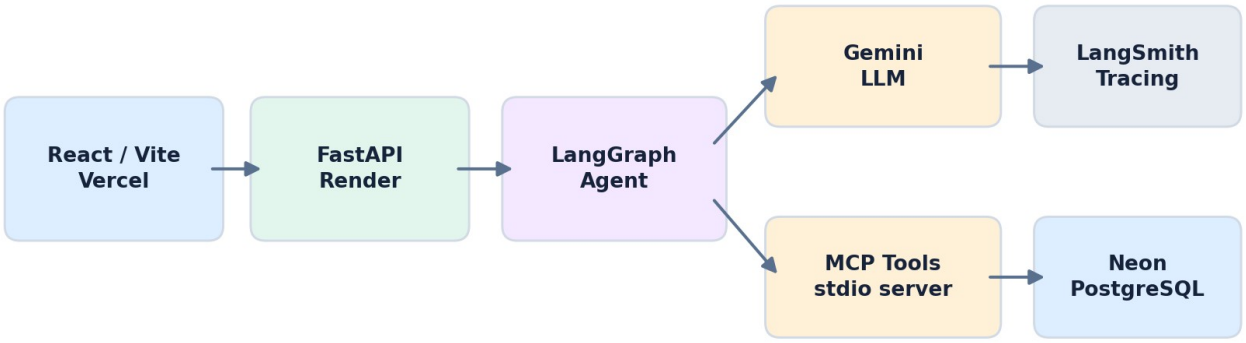
2. Technology stack

Layer	Technology	Why we used it
Frontend	React, Vite, Tailwind CSS, React Router, Axios	Fast single-page UI, routing, API calls and responsive styling
Backend	Python 3.14, FastAPI, Pydantic, SQLAlchemy	API endpoints, input validation and database

		operations
Database	Neon PostgreSQL, psycopg	Persistent cloud storage for accounts, transactions, auth and checkpoints
AI model	Google Gemini via langchain-google-genai	Understands questions and decides when tools are needed
Agent workflow	LangGraph + PostgreSQL checkpointer	Controls chat flow, approvals and conversation persistence
Tool interface	MCP server + langchain-mcp-adapters	Makes financial functions callable by the agent
Observability	LangSmith	Shows model calls, tool inputs/outputs and errors
Deployment	Vercel (frontend), Render (backend), Neon (DB)	Hosted web app accessible from browser and phone
Development	uv, Git and GitHub	Dependency management, version control and deployment source

3. Overall system architecture

The browser loads the React application from Vercel. React sends API requests through a same-origin /api proxy to the FastAPI service on Render. FastAPI handles dashboard requests directly and passes chat requests to LangGraph. LangGraph can call Gemini and MCP tools. Financial records live in Neon PostgreSQL.



Why use the /api proxy?

The frontend sets VITE_API_BASE_URL=/api. Vercel rewrites requests such as /api/chat/ to the Render backend route /chat/. This keeps browser requests on the frontend origin and helps cookie-based login work reliably. A separate rewrite serves index.html for React routes such as /login and /dashboard.

4. Data model: accounts and transactions

Each bank account has a name, opening balance, currency and unique ID. Every transaction belongs to an account. Transaction types include expense, income, transfer_in and transfer_out.

Concept	How it works
Current balance	Opening balance + income + transfer_in − expenses − transfer_out
Multiple accounts	Transactions reference an account_id; summaries aggregate across accounts
Transfers	A transfer creates paired transfer_out / transfer_in records with a shared transfer ID
Categories	Expenses can be grouped into categories; missing categories can appear as Other
Idempotency	A unique key helps prevent accidental duplicate transaction writes

Example: an account starts at ₹25,000, receives ₹700, spends ₹2,300 and transfers ₹3,000 out. Its current balance is ₹20,400. The backend calculates this value; the AI should not invent it.

5. Main features and screens

Dashboard

The dashboard shows combined and account-level balances, recent transactions and category spending. It calls protected FastAPI endpoints, including /summary/balances, /summary/categories and /transactions/.

Chat assistant

The chat page supports questions such as “What is my total balance?”, “How much did I spend this month?” and “Which category has the highest spending?”. The assistant selects the relevant tool and explains its result in plain language.

Transaction workflow

The assistant can propose recording an expense or income. MoneyMate requires explicit approval before the write occurs. Transfers and account creation are handled by the existing backend routes; chatbot-based account creation was discussed but not yet confirmed implemented.

6. What MCP does in MoneyMate

MCP (Model Context Protocol) is the bridge between the AI agent and financial functions. Rather than giving Gemini direct database access, MoneyMate exposes a small set of controlled tools from a Python MCP server. The agent discovers these tools using MultiServerMCPClient, launching the server as a local stdio subprocess inside the Render backend.

MCP tool	Purpose
get_account_balances	Read all account balances and the combined total
get_monthly_expenses	Read current-month spending total
get_category_spending	Read category totals for day, week or month
add_transaction	Record an expense or income after approval

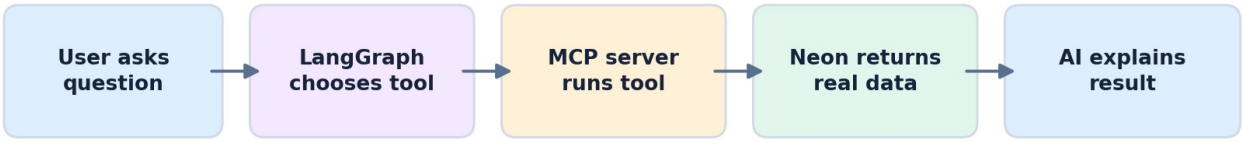


Figure 2. Example read flow: a balance question triggers the MCP tool, which reads Neon data and returns structured results. Gemini then turns those results into a human-friendly answer.

Example: “What is my total balance?”

- React sends the message to POST /chat/.
- LangGraph passes the conversation to Gemini with available tool definitions.
- Gemini requests get_account_balances.
- The MCP server opens a SQLAlchemy session and calculates current balances from Neon.
- The tool returns account balances and total_balance; Gemini summarizes them.

Important distinction: MCP exposes callable capabilities; LangGraph orchestrates when they run; Gemini interprets user intent. The database and backend remain the source of truth for money values.

7. How LangGraph controls the agent

LangGraph is used because MoneyMate needs more than a single question-answer model call. It maintains conversation state, executes tools and pauses sensitive operations for human approval.

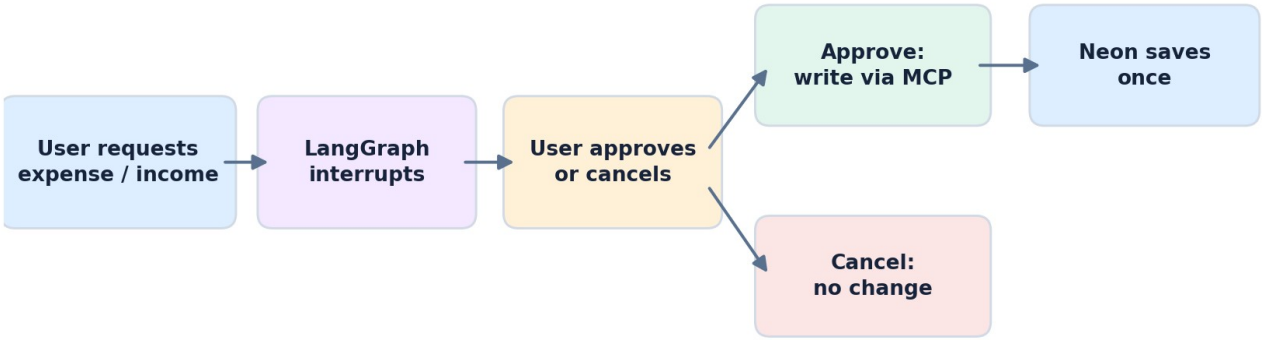


Figure 3. A write action is paused until the user explicitly approves. Cancellation makes no database change.

Approval and duplicate protection

For add_transaction, LangGraph interrupts before executing the write. After approval, the tool writes through the backend and returns a confirmed result. A tool-call-derived idempotency key and a database uniqueness rule prevent the same request from creating duplicate transactions. Confirmation and cancellation messages can be generated in Python without a second Gemini call.

Conversation persistence

AsyncPostgresSaver stores LangGraph checkpoints in PostgreSQL. The frontend remembers a conversation ID and calls /chat/history/{conversation_id} on reload. This restores previous messages and pending approval state when correctly configured.

8. Authentication and API security

MoneyMate uses a single-owner password-based login. Password verification uses Argon2. On login, the backend creates a database-backed session and sends an HTTP-only cookie. State-changing requests require a matching CSRF token. React restores sessions with /auth/me and attaches the token to protected requests.

Protection	Purpose
Argon2 password hash	Avoid storing the login password in plaintext
HTTP-only secure cookie	Keep the session token out of frontend JavaScript
CSRF token	Protect state-changing API requests
Protected FastAPI routes	Require a valid session for accounts,

	transactions, summaries and chat
Idempotency key	Prevent duplicate financial writes

Deployment note: COOKIE_SECURE=true is used for HTTPS. Login rate limiting and origin-validation hardening were discussed, but their final production verification was not confirmed in the project walkthrough.

9. Where and how MoneyMate is deployed

Component	Hosting	Configuration
React frontend	Vercel	Vite build; frontend/ root; dist output
FastAPI + LangGraph + MCP	Render Web Service	backend/ root; uv dependencies; Uvicorn start command
Financial data	Neon PostgreSQL	DATABASE_URL
LangGraph checkpoints	Neon PostgreSQL	CHECKPOINT_DATABASE_URL
AI	Google Gemini API	GOOGLE_API_KEY
Traces	LangSmith	LANGSMITH_TRACING, LANGSMITH_API_KEY, LANGSMITH_PROJECT

Deployment flow

Push code to GitHub → Vercel builds the React frontend and Render builds the Python backend → Render starts FastAPI and the MCP subprocess → both use Neon for persistent data → LangSmith receives tracing events.

Live frontend: https://moneymate-personal.vercel.app

Backend: https://moneymate-backend-4jwh.onrender.com

Health endpoint: /health; interactive API documentation: /docs

Free-hosting caveat: Render may sleep during inactivity, so the first request after a quiet period can be slower. Database data remains in Neon, not on Render's local disk.

10. Problems we encountered and how we fixed them

Problem	What we learned / fixed
Backend failed on Render	Corrected malformed Neon checkpoint connection URL query parameters
Vercel API calls returned 404/405	Adjusted rewrite rules so /api requests

	reach FastAPI and SPA fallback does not intercept them
Chat said it could not read balances	LangSmith exposed a connect_timeout connection error; database configuration was corrected
Chat returned old account values	MCP subprocess was using SQLite instead of Neon; passed DATABASE_URL explicitly and removed silent SQLite fallback
Chat disappeared on refresh	Restored conversation history using persistent checkpoints and the conversation ID
No LangSmith traces	Fixed LangSmith API/workspace configuration after 403 Forbidden ingest errors

11. A practical walkthrough

Scenario A: Checking spending. The user asks “How much did I spend this month?” → Gemini selects get_monthly_expenses → the MCP tool queries the category summary → the backend returns a verified total → the assistant explains it.

Scenario B: Recording an expense. The user says “Add a ₹250 food expense to HDFC” → the agent resolves the account → displays approval → on approval, add_transaction runs with an idempotency key → Neon saves the transaction → the assistant confirms the result and the dashboard reflects it.

Scenario C: Reloading a chat. The user refreshes the Vercel page → React restores login via /auth/me → loads /chat/history/{conversation_id} → renders messages from PostgreSQL checkpoints.

12. Key design lessons

- Use AI for language understanding, not as the source of financial truth.
- Keep data access inside small, auditable MCP tools.
- Use LangGraph for approval, state and safe tool execution.
- Persist business data and checkpoints in PostgreSQL, not local deployment files.
- Treat model-generated explanations of failures as hypotheses; inspect tool traces.
- Keep credentials in hosting environment variables, never in Git.
- Use an API proxy when cookie-based frontend and backend hosting differ.

13. Current scope and next improvements

Implemented and tested in the project walkthrough: multi-account balances, spending summaries, transaction entry with approval, persistent chat, authentication, Neon-backed MCP data access and cloud deployment.

Potential next improvements: create bank accounts through an approved MCP write tool; add a dashboard “Add Account” form; verify rate limiting and remaining production security checks; add automated tests for financial calculations and deployment smoke tests; and add LangSmith evaluations for numerical answer quality.

14. Quick glossary

Term	Simple meaning
FastAPI	The Python web server that exposes application endpoints
MCP	A standard way for an AI system to call external tools
LangGraph	The workflow engine controlling the agent and its state
Checkpoint	A saved snapshot of conversation/workflow state
Neon	Cloud-hosted PostgreSQL database
Vercel	Hosting platform for the React frontend
Render	Hosting platform for the FastAPI backend
LangSmith	A debugging view of model and tool execution

Bottom line: MoneyMate combines a normal, dependable finance application with an AI interface. React displays information, FastAPI validates and serves it, PostgreSQL stores it, MCP exposes controlled financial tools, and LangGraph manages AI interactions safely.